

01211373 Machine Learning and Programming for Industry

LEARNING FROM REWARD

Reinforcement Learning

Week 12 — Policies Nobody Wrote Down

Q-learning • DQN • Ch. 19

Last Week → This Week

WEEK 11 RECAP

- Attention reads any position directly
- One equation: softmax($QK^T/\sqrt{d}$)V
- At L=300 it beat the LSTM eight-fold
- Every model so far was handed the right answer

TODAY

- No labels at all — only a scalar reward
- Markov decision processes
- Q-learning: one update rule
- Exploration vs exploitation
- Deep Q-networks for continuous state

A Different Kind of Learning



Supervised

Here is an image.
Here is its label.

Learn the mapping.

Weeks 1–11



Unsupervised

Here is some data.
No labels at all.

Find the structure.

Week 5



Reinforcement

Here is a world.
Actions change it.
Some outcomes score.

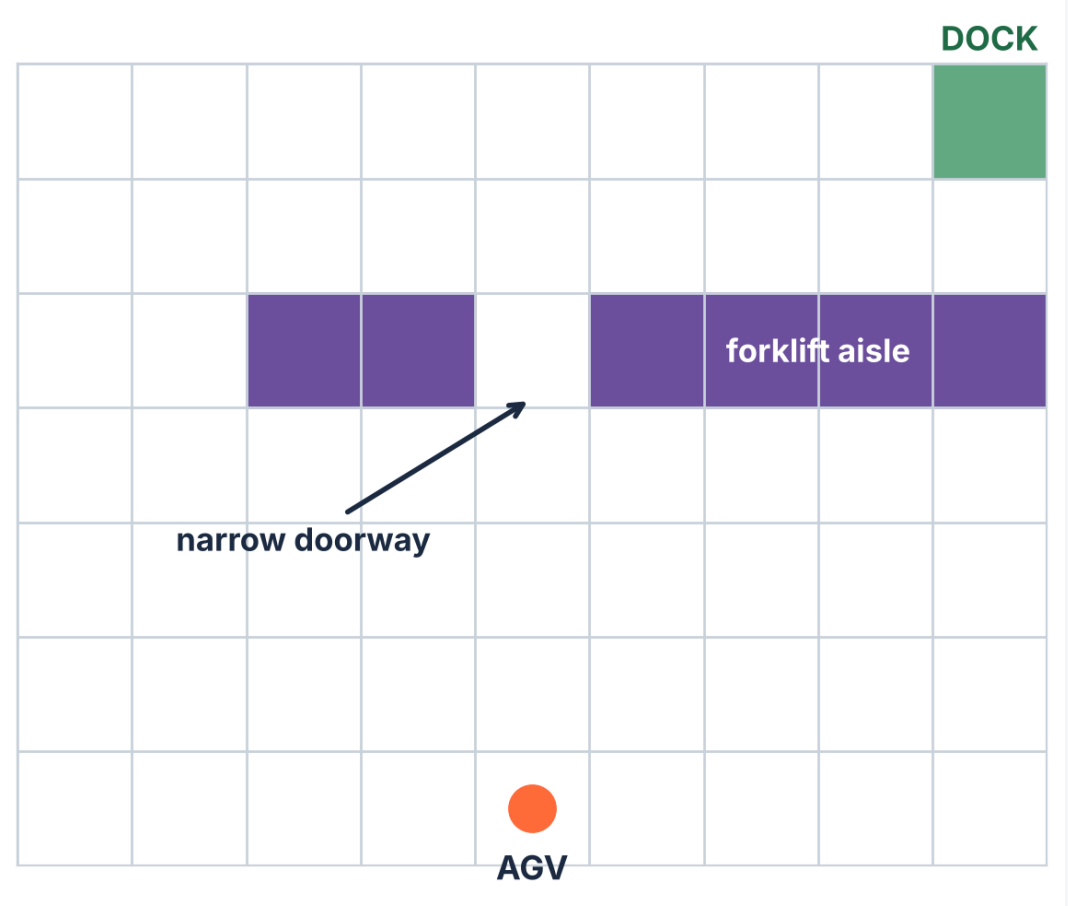
Find a policy.

Today

The catch: nobody tells the agent which action was right. It only learns that some sequence of actions eventually paid off — and has to work backwards from there.

Today's Problem: An AGV on the Factory Floor

Reach the loading dock. Nobody provides a route — only a score at the end of each attempt.



+10

reach the loading dock

−10

enter the forklift aisle

−0.1

every move — time costs money

That is the entire specification. No route, no waypoints, no clearance rule.

The Framework: a Markov Decision Process



Every RL problem is written down the same way. Getting this framing right is most of the engineering work.

States S	where the system can be	the 63 cells of the floor
Actions A	what the controller may do	up, down, left, right
Reward $r(s,a)$	what we actually want	+10 dock, −10 aisle, −0.1 per move
Discount γ	how much the future counts	0.95 — patient, but not infinitely

"Markov" means the current state is enough. If your state leaves out something that matters — load, temperature, wear — no RL algorithm can recover it.

The Action-Value Function Q(s, a)

Q(s, a) is the total future reward you expect if you take action a in state s, then act greedily from then on.

$$\pi(s) = \text{argmax over a of } Q(s, a)$$

If you know Q, you have a controller: in every state, take the action with the largest value.

It is a table you can read

63 states × 4 actions. Print it, plot it, argue with it — unlike most learned controllers.

It is also a trap

63 states is a toy. Any real state space is continuous, and the table disappears.

Q-Learning: One Update Rule

$$Q(s,a) := Q(s,a) + \alpha\,[\, r + \gamma \cdot \max Q(s',a') - Q(s,a)\,]$$

$r + \gamma \max Q(s',\cdot)$

THE NEW ESTIMATE
the reward you just got, plus the discounted value of wherever you landed

$- Q(s,a)$

THE OLD ESTIMATE
what you believed before taking the action

$\alpha\,[\ \ldots\ ]$

THE CORRECTION
move the old estimate part of the way toward the new one

The bracket is the temporal-difference error. It is the whole algorithm: act, see what happened, and close the gap between what you expected and what you got.

One trap: at a terminal state there is no future — the target is just r. Forget this and the agent credits a dead state with value forever.

Exploration vs Exploitation



Act greedily and you keep collecting the reward you already know about. Explore and you pay, in the hope of finding something better.

EXPLOIT

Take the best action you currently know. Safe, and possibly permanently second-rate.

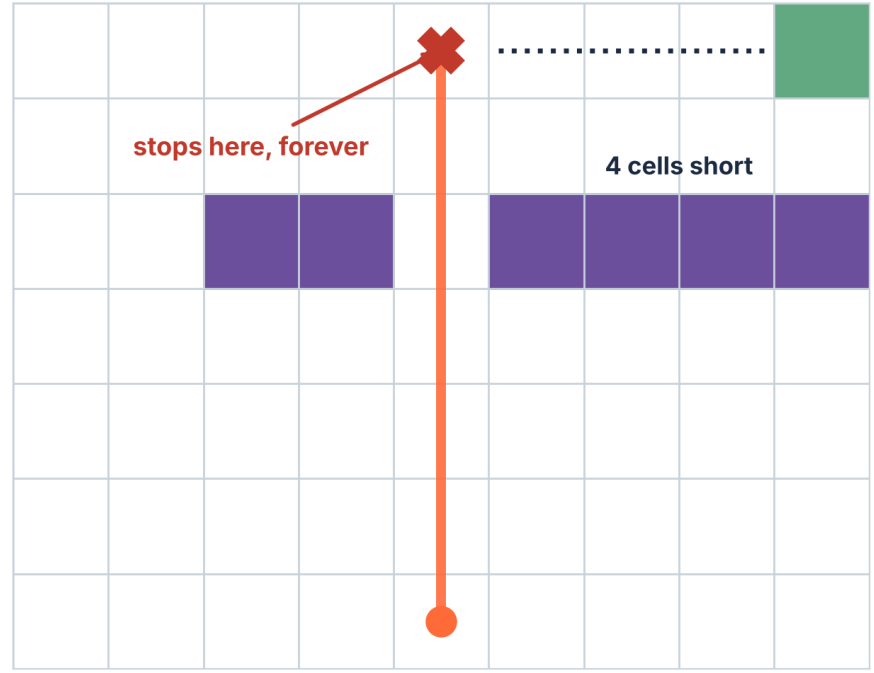
EXPLORE

Try something else. Costs reward now; it is the only way to discover a better route.

ε-greedy: act greedily with probability 1 − ε, act at random with probability ε

The textbook claim: set ε = 0 and the agent gets stuck on the first adequate route it finds. We tested it in the lab — and it is not that simple.

Measured: Where Exploration Actually Comes From



ε = 0 with pessimistic initial values

initial Q	ε	success
0	0	100%
0	0.2	100%
−5	0	0%
−5	0.2	100%

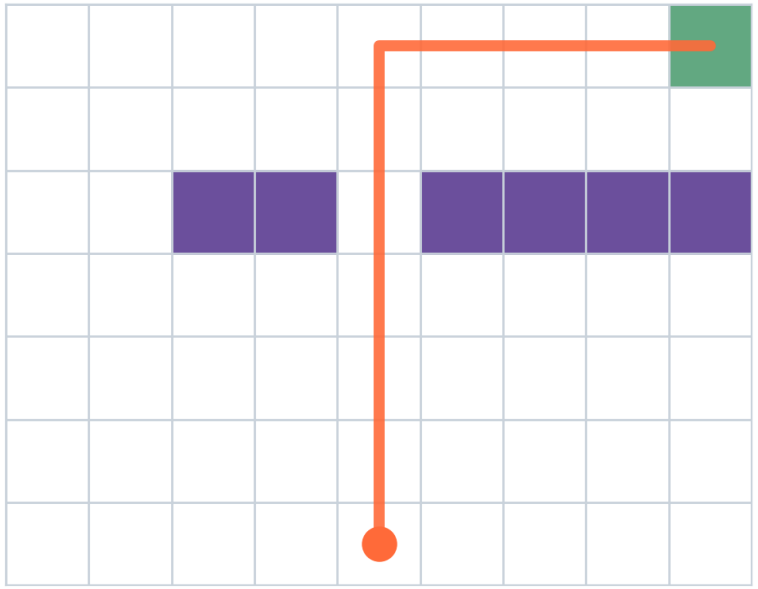
Zero init is already optimistic. With a −0.1 step cost, anything tried drops below zero while anything untried still sits at zero. The agent explores without being told to.

Why it froze: pushing into a wall costs only −0.1 and settles at -2.00; every action it never tried still looks like -5.00. It stopped four cells from the dock — not confused, but confident and wrong.

The Decisive Experiment: Change the Wheels

Same grid, same rewards, same algorithm. The only change is a 20% chance the AGV slips sideways.

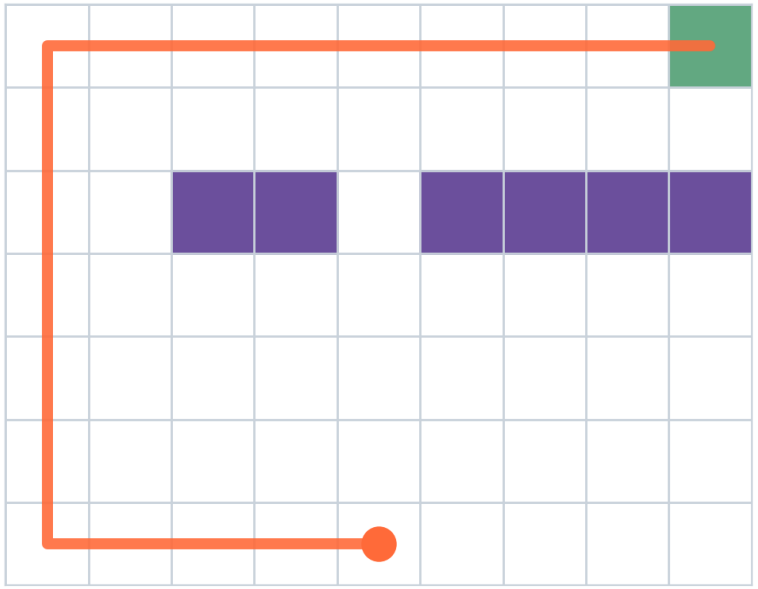
Perfect traction



10 steps through the doorway

9.10 return, doorway route

20% wheel slip



18 steps the long way round

7.72 return, detour route

Nobody wrote a clearance rule. The agent inferred it from the reward: once you might not go where you aimed, squeezing through a gap stops being worth it.

Is the Detour Better, or Merely Different?

A fair challenge — so we forced the agent down the doorway under slip and measured the cost.

policy	wheel slip	return	success
doorway (forced)	none	9.10	100%
doorway (forced)	20%	4.41	78%
learned detour	20%	7.72	100%

Forced through the doorway under slip, the AGV crashes into the forklift aisle in 22% of runs.

This is the engineering lesson: a path planner that assumes perfect actuation will happily route a real robot into a wall. The detour is not timid — it has the higher expected return.

When the Table Runs Out

The Q-table worked because the floor had 63 states. Now balance a pole on a cart — a plant you already know.

4 continuous variables

cart position, cart velocity,
pole angle, pole rate

20 bins each = 160,000 states

a crude discretisation, and
most cells never visited

Replace the table with a network

Q(s,a) becomes a function that
generalises between nearby states

That is a Deep Q-Network. The update rule does not change at all — only how Q is stored.

Two Tricks That Make It Train

A neural Q-function is unstable on its own. Two additions fix it, and both address the same thing: the target must hold still.

Replay buffer

Store transitions and learn from random past samples.

Without it: consecutive steps are highly correlated, and the network chases its own tail.

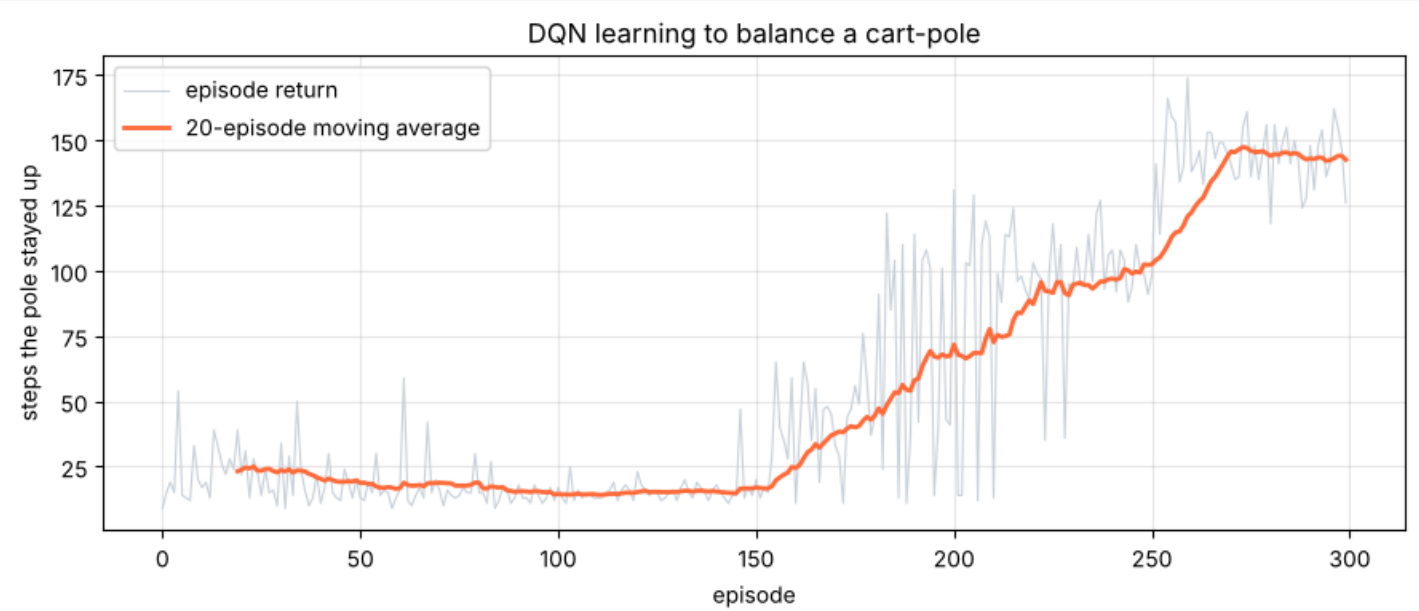
Target network

A frozen copy of the network computes the target, synced every 50 episodes.

Without it: the target moves every time you update, and training can diverge.

The target: $y = r + \gamma (1 - \text{done}) \cdot \max Q_{\text{target}}(s', a')$. The $(1 - \text{done})$ factor is the terminal-state rule from slide 7, written as arithmetic.

Measured: DQN on the Cart-Pole



21.1

steps survived,
first 50 episodes

143.4

steps survived,
last 50 episodes

29 s

total training time
on a laptop CPU

about a sevenfold improvement

Read the curve honestly. It is flat until episode 150 — exactly where ε finishes annealing — then climbs and plateaus well short of mastery. Training longer does not fix it; 500 episodes gives the same result.

RL and Classical Control: an Honest Comparison

	classical control (LQR, MPC)	reinforcement learning
needs a model	yes — that is the starting point	no — only interaction
needs data	almost none	a great deal
stability guarantees	yes, provable	essentially none
handles contact, friction, things you cannot model	poorly	this is where it earns its keep
cart-pole	an afternoon, with a proof	29 s of training, still not mastered

Use RL where you have no model and data is cheap. If you can write the dynamics down, write them down — you will get a better controller faster.

This Week's Lab



Everything on these slides, measured by you. The whole notebook runs in a couple of minutes on a CPU.



Implement the Q-learning update

Two lines, including the terminal case



Run the exploration experiment

Find out where exploration really comes from



Prove the detour is better

Force the doorway route and measure the crashes



Build a DQN

ε-greedy action selection and the TD target

Next week: Deployment & Sim-to-Real — everything that breaks when a model leaves your laptop.

What to Take Away

1

RL learns a policy from reward alone

No labels. The reward function is your entire specification — and writing it badly is the most common failure.

2

Q-learning is one update rule

Act, observe, close the gap between what you expected and what you got. Everything else is bookkeeping.

3

Exploration must come from somewhere

ε-greedy, optimistic initialisation, or something else. Remove it all and the agent froze four cells from the goal.

4

Uncertainty changes the optimal policy

The slipping AGV chose an 18-step detour over a 10-step doorway — and avoided a 22% crash rate by doing so.

5

A learned policy is only valid where it has data

Off the training distribution, you are reading numbers that were never learned.